

PROYECTO INTERMODULAR

CICLO FORMATIVO DE GRADO SUPERIOR EN DESARROLLO APLICACIONES MULTIPLATAFORMA



PISPAX

Adrián Salazar Nicolas

Tutor: Andrés Dolón Sevilla

DECLARACIÓN JURADA DE AUTORÍA

D. / Dña _Adrian Salazar Nicolas_ con DNI _53807018J_

como autor de este documento académico, titulado: _PISPAX APP de Transporte_

y presentado como Proyecto Intermodular para la obtención del título oficial de Técnico Superior en _Desarrollo Aplicaciones Multiplataforma__

DECLARO QUE

Soy el único autor del trabajo, con la excepción de referencias a contenidos o ideas de otros autores, en cuyo caso han sido explícitamente citados.

El trabajo remitido es un documento original, que no ha sido publicado, ya sea total o parcialmente, ni presentado para obtención de un título académico en ninguna institución académica u organización.

No he trasgredido ninguna norma del CEFP con respecto al plagio ni a las leyes establecidas que protegen la propiedad intelectual.

Soy consciente de que el hecho de no respetar estos extremos es una falta grave de integridad académica y podrá ser objeto de sanciones.

En _____, a _____ de _____ de 2026

(insertar firma)

Fdo:

RESUMEN

PisPax es una aplicación móvil diseñada para conectar clientes que necesitan servicios de transporte de mercancías con transportistas que disponen de vehículos con capacidad infrautilizada. El sistema gestiona el ciclo completo del viaje: solicitud, asignación, seguimiento mediante estados y finalización del servicio. Desarrollada en Android con Java sobre una arquitectura cliente-servidor REST, el backend se implementa con Spring Boot 3 y base de datos MySQL 8. El MVP se centra en las funcionalidades esenciales: autenticación con JWT, gestión de vehículos, creación y aceptación de viajes, y trazabilidad mediante ocho estados operativos definidos. La plataforma aplica los principios de un TMS simplificado y escalable, inspirada en modelos de éxito como OnTruck o Uber aplicados al sector de mercancías por carretera.

Palabras clave: transporte de mercancías, aplicación móvil, Android, Spring Boot, marketplace logístico, MVP, TMS, Java

ABSTRACT

PisPaxp is a mobile application designed to connect clients requiring freight transport services with carriers that have underutilised vehicle capacity. The system manages the complete trip lifecycle: request, assignment, state-based tracking, and service completion. Developed natively for Android using Java, the application follows a REST client-server architecture with a Spring Boot 3 backend and a MySQL 8 relational database. The MVP focuses on core functionality: JWT-based authentication, vehicle management, trip creation and acceptance, and traceability through eight defined operational states. The platform applies the principles of a simplified and scalable Transportation Management System (TMS), drawing inspiration from successful digital intermediation models such as OnTruck and Uber adapted to the road freight sector.

Key words: freight transport, mobile application, Android, Spring Boot, logistics marketplace, MVP, TMS, Java

1 Tabla de contenido

1. INTRODUCCIÓN	5
1.1 Introducción General	5
1.2 Justificación del proyecto	6
1.3 Objetivos	6
1.4 Contextualización	7
1.5 Motivación	7
2 Antecedentes	8
3 METODOLOGÍA.....	9
3.1 Tipo de investigación y enfoque	9
3.2 Metodología de desarrollo	9
3.3 Procedimiento y fases del proyecto	10
4 DESARROLLO	11
4.1 Análisis del sistema	12
4.1.1 Especificación de requisitos	12
4.2 Tecnologías utilizadas	16
4.2.1 Aplicación Android (Frontend)	16
4.2.2 API REST (Backend).....	17
4.2.3 Base De Datos.....	18
4.2.4 Comunicación Cliente-Servidor	18
4.2.5 Resumen y justificación del stack.....	18
4.3 Diseño del sistema	19
4.3.1 Arquitectura de la aplicación.....	19
4.3.2 Diseño de la base de datos.....	21
4.3.3 Diseño de la interfaz de usuario (UI)	30
4.4 Implementación.....	31
4.5 Pruebas realizadas.....	31
5 ANÁLISIS	32
5.1 Análisis del cumplimiento de objetivos	32
5.2 Evaluación del sistema desarrollado	32
5.3 Comparación con soluciones existentes.....	32
5.4 Limitaciones del proyecto.....	32
6 CONCLUSIONES	32
7 Manual Para Usuario / Documentación Caso Practico	32
8 BIBLIOGRAFÍA.....	32
9 ANEXOS	32

2 Índice de Ilustraciones

Ilustración 1.Esquema	19
Ilustración 2.Arquitectura	20
Ilustración 3.Tabla de compatibilidad	30
Ilustración 4.Paleta de Colores.....	30
Ilustración 5.Workflow	31

3 Indice Tablas

Tabla 1 Requisitos funcionales del sistema.....	12
Tabla 2 Requisitos no funcionales del sistema.....	14
Tabla 3.Casos de uso principales.....	15
Tabla 4.Stack	17
Tabla 5.Stack Back	17
Tabla 6.DB	18
Tabla 7.Justificacion del Stack	18
Tabla 8.Tabla Usuario.....	21
Tabla 9.Tabla Vehiculo	22
Tabla 10.Tipo Mercancia	23
Tabla 11.Mercancia-Vehiculo.....	23
Tabla 12.Viaje	24
Tabla 13.Rol Usuarios.....	25
Tabla 14.Tipo Vehiculos	25
Tabla 15.Carnet Requerido	26
Tabla 16.Estado Viaje	26
Tabla 17.Relacion de Tablas	27
Tabla 18.Datos Semillas para MVP	27

1. INTRODUCCIÓN

3.1 Introducción General

El proyecto consiste en el desarrollo de una aplicación móvil orientada a empresas y profesionales del transporte que disponen de una flota de furgonetas, furgones o vehículos industriales que no están siendo utilizados durante parte del día, ya sea por falta de trabajo o por inactividad operativa. La app permitirá que estos vehículos puedan realizar servicios adicionales de transporte bajo demanda, generando ingresos extra para sus propietarios.

Además, la plataforma incluirá la posibilidad de que pequeños autónomos o particulares con vehículo propio se registren como transportistas, siempre que cumplan determinados requisitos legales y técnicos. Todo el proceso de contratación, asignación de viajes y pago se gestionará íntegramente a través de la aplicación, lo que garantiza trazabilidad, evitar trabajos fuera de la plataforma y aporta seguridad jurídica y operativa al servicio.

Este proyecto se alinea con la tendencia actual de digitalización del sector logístico, donde la mayor parte de la captación de clientes y la contratación de servicios se realiza a través de aplicaciones móviles simples, rápidas y basadas en modelos cloud escalables.

3.2 Justificación del proyecto

El mercado del transporte está migrando hacia modelos digitales similares a OnTruck ,Uber, Cabify , Blablacar ,QBK o Yako.

Pero en el sector de mercancías aún existe una gran oportunidad de optimización. Muchas empresas infrutilizan sus vehículos durante varias horas al día, lo que supone un coste operativo elevado y una pérdida de potencial económico.

Por otro lado, los usuarios de servicios logísticos demandan inmediatez, trazabilidad, pagos integrados y una experiencia móvil sencilla. Por ello, el uso de arquitecturas cloud resulta imprescindible para garantizar escalabilidad, disponibilidad y rendimiento sin necesidad de desplegar infraestructura propia.

Este proyecto es adecuado como Trabajo de Fin de Ciclo en el ámbito del Desarrollo de Aplicaciones Multiplataforma porque permite trabajar aspectos reales del sector tecnológico: conexiones en tiempo real, gestión de pagos, diseño de APIs, bases de datos complejas, despliegue sobre servicios cloud y diseño UI/UX funcional. Todo ello representa un caso práctico completo y comparable a un producto real en el mercado.

3.3 Objetivos

Objetivo general

Desarrollar una aplicación móvil funcional que permita la gestión de servicios de transporte entre empresas, autónomos y clientes, optimizando el uso de vehículos disponibles mediante un sistema digital centralizado y seguro.

Objetivos específicos

1. Implementar un sistema de creación de viajes óptimos entre un punto A y un punto B.
2. Integrar un modelo de seguridad que garantice la correcta entrega de mercancías.
3. Desarrollar una arquitectura escalable basada en servicios cloud para soportar incrementos futuros de demanda.
4. Incorporar un sistema de pagos interno que permita realizar cobros y liquidaciones desde la app.
5. Diseñar una interfaz gráfica moderna, sencilla y orientada tanto a clientes como a transportistas.
6. Gestionar correctamente la base de datos de usuarios, vehículos, viajes y operaciones mediante un backend eficiente.
7. Geolocalización de la mercancía en el momento de reparto, con sus diferentes estados.

Tipo de Tiempos

- 1.Salida hacia la recogida
- 2.Llegada a la recogida
- 3.Mercancia recogida
- 4.Salida hacia la entrega
- 5.Llega a la entrega
- 6.Mercancia descargada

3.4 Contextualización

El sector del transporte de mercancías por carretera ha experimentado, durante décadas, una evolución lenta en comparación con otros sectores económicos. Tradicionalmente, los procesos de contratación, planificación y seguimiento de envíos se han basado en métodos poco digitalizados, lo que ha dificultado la optimización de recursos y la mejora de la eficiencia operativa. Esta situación ha generado un escenario propicio para la aparición de nuevas soluciones tecnológicas orientadas a modernizar el sector.

En el contexto de España y Europa, el transporte terrestre de mercancías juega un papel fundamental en la economía, siendo uno de los principales medios para la distribución de productos. La cercanía geográfica y el elevado volumen de transporte convierten a estos mercados en un entorno viable para la implantación de plataformas digitales que faciliten la conexión entre clientes y transportistas, mejorando la accesibilidad y competitividad del servicio.

Desde el punto de vista tecnológico, el uso de infraestructuras basadas en la nube se ha consolidado como un estándar para el desarrollo de aplicaciones modernas. Los modelos cloud permiten a pequeñas y medianas empresas lanzar productos al mercado sin necesidad de realizar grandes inversiones iniciales en infraestructura propia, ofreciendo escalabilidad, disponibilidad y flexibilidad conforme crece la demanda del servicio.

Por otro lado, los usuarios de servicios de transporte demandan cada vez más soluciones que ofrezcan precios ajustados y transparentes, adaptados a cada situación concreta, así como un mayor control sobre el estado de la mercancía. Aspectos como la trazabilidad, el seguimiento en tiempo real y la agilidad en la contratación del servicio se han convertido en elementos clave dentro de la experiencia del cliente, impulsando la adopción de aplicaciones móviles como principal canal de interacción.

3.5 Motivación

La elección de este proyecto surge del interés por comprender en profundidad cómo se desarrollan aplicaciones móviles modernas que funcionan en entornos reales y resuelven necesidades concretas del mercado. El sector del transporte y la logística vive un proceso de transformación digital, donde las aplicaciones se han convertido en la herramienta principal para conectar oferta y demanda de forma eficiente.

Una de las principales motivaciones detrás del proyecto es la **oportunidad de optimizar recursos infrautilizados**. Aunque ya existen aplicaciones móviles complejas, como los sistemas TMS, todavía hay un número considerable de flotas y transportistas particulares cuyos vehículos permanecen inactivos durante largos periodos, sin generar ingresos. La aplicación propuesta busca **atraer a este público objetivo**, ofreciendo una plataforma sencilla y accesible que permita rentabilizar estos vehículos de manera segura y eficiente. Se considera que, además de un desarrollo técnico sólido, será clave un **buen enfoque de marketing** para captar y fidelizar a este segmento de usuarios.

Desde la perspectiva del desarrollador, este proyecto representa una oportunidad para trabajar con tecnologías presentes en la realidad laboral actual: infraestructura cloud, sistemas de tiempo real, bases de datos distribuidas, pasarelas de pago e interfaces móviles orientadas al usuario. Además, permite crear un producto con potencial de evolución y lanzamiento al mercado, lo que aporta un valor añadido al portfolio profesional.

En definitiva, la motivación principal es desarrollar una aplicación que no solo cumpla un fin académico, sino que pueda convertirse en un proyecto real, escalable y con utilidad práctica en el sector logístico, generando un **impacto tangible** en la eficiencia de transportistas y clientes.

4 Antecedentes

El mercado del transporte de mercancías por carretera está experimentando una transición hacia **modelos digitales** inspirados en plataformas tipo Uber, Ontruck o Blablacar. Sin embargo, en este sector aún existe una gran oportunidad de optimización, ya que muchas empresas infrautilizan sus vehículos durante varias horas al día, lo que supone un coste operativo elevado y una pérdida de potencial económico.

En este contexto, la aplicación propuesta se basa en los principios de un **Transportation Management System (TMS)**, adaptados a un entorno simplificado y orientado a la intermediación entre clientes y transportistas. De esta forma, la app incorpora funcionalidades clave de un TMS tradicional, como la planificación de viajes óptimos, la gestión de flotas, la trazabilidad de la mercancía y el control de pagos, pero enfocadas a un público más amplio y con necesidades de uso sencillo y accesible desde dispositivos móviles.

El uso de la arquitectura cloud resulta esencial para garantizar **escalabilidad, disponibilidad y rendimiento**, permitiendo ofrecer una solución sin necesidad de desplegar infraestructura propia. Este enfoque permite que tanto empresas con flotas infrautilizadas como autónomos con vehículos propios puedan integrar sus servicios en la plataforma, optimizando recursos y generando ingresos adicionales.

Este proyecto es adecuado como Trabajo de Fin de Ciclo en el ámbito del Desarrollo de Aplicaciones Multiplataforma porque permite trabajar con aspectos reales del sector tecnológico: conexiones en tiempo real, gestión de pagos, diseño de APIs, bases de datos complejas, despliegue sobre servicios cloud y diseño UI/UX funcional. Todo ello representa un caso práctico completo y comparable a un producto real en el mercado, como OnTruck, y demuestra cómo los conceptos de un TMS pueden aplicarse de manera efectiva en una solución digital para el transporte de mercancías

5 METODOLOGÍA

5.1 Tipo de investigación y enfoque

El presente proyecto se enmarca dentro de una investigación aplicada, ya que tiene como objetivo principal la resolución de un problema real existente en el sector del transporte de mercancías por carretera. A diferencia de la investigación teórica, este trabajo se centra en el diseño y desarrollo de una solución tecnológica que permita mejorar la eficiencia, la transparencia y la accesibilidad en la contratación de servicios de transporte para mercancías.

Asimismo, se trata de una investigación de carácter tecnológico, dado que la propuesta se basa en la utilización de herramientas y tecnologías software para la creación de una aplicación orientada a la gestión, seguimiento y trazabilidad de mercancías. El enfoque práctico del proyecto permite aplicar conocimientos adquiridos durante la formación académica a un contexto real, evaluando su viabilidad y utilidad en un entorno empresarial como el caso pensado una Startup.

5.2 Metodología de desarrollo

El proyecto adopta un **enfoque metodológico mixto**, con predominio del **análisis cualitativo**, orientado al diseño y desarrollo de una solución tecnológica real centrada en las necesidades de los usuarios. Este enfoque permite estudiar el funcionamiento del sector del transporte de mercancías, analizar plataformas similares existentes en el mercado y definir una aplicación móvil funcional, escalable y viable desde un punto de vista técnico y empresarial.

Uno de los pilares fundamentales del enfoque metodológico es la **experiencia de usuario (UX)**. Dado que se trata de una aplicación móvil, se considera prioritario ofrecer una interfaz sencilla, intuitiva y rápida, que facilite la contratación del servicio sin fricciones innecesarias. El diseño de la aplicación se plantea desde un enfoque centrado en el usuario, analizando sus necesidades, expectativas y comportamiento, con el objetivo de mejorar la usabilidad y la satisfacción durante el uso de la plataforma.

En el análisis de necesidades se identifican dos perfiles principales de usuario:

- **Cliente demandante del servicio de transporte**, cuya principal necesidad es la rapidez en la contratación del servicio, la sencillez de uso de la aplicación, la fiabilidad del transportista y la seguridad en el envío de su mercancía. Este usuario busca una experiencia similar a las aplicaciones de tipo *Uber-like*, donde el proceso sea ágil, transparente y accesible desde el dispositivo móvil.
- **Transportista o proveedor del servicio**, ya sea una empresa con flota de vehículos infrutilizados o un autónomo con vehículo propio que cumpla los requisitos establecidos. Para este perfil, la aplicación debe ofrecer facilidad en la aceptación de viajes, claridad en la información del servicio, seguridad en los pagos y una gestión eficiente de los trayectos, permitiéndole optimizar el uso de sus recursos y generar ingresos adicionales.

Desde el punto de vista tecnológico, el proyecto prioriza la simplicidad, la mantenibilidad y la viabilidad, evitando la incorporación de tecnologías complejas que no aporten un valor directo

en la fase inicial de la aplicación. En este sentido, aunque se ha investigado el uso de técnicas de inteligencia artificial en aplicaciones similares, especialmente en el cálculo dinámico de precios, se ha decidido no incluir esta tecnología en el alcance actual del proyecto, dejándola como posible línea de evolución futura.

El contexto actual del sector del transporte de mercancías, junto con el auge de plataformas digitales de intermediación, refuerza la validez de este enfoque metodológico. La aceptación de modelos similares en otros ámbitos, como el transporte de personas, demuestra que una solución digital bien diseñada puede mejorar la eficiencia del servicio y la experiencia de los usuarios, justificando así el enfoque adoptado en este proyecto.

5.3 Procedimiento y fases del proyecto

El desarrollo de la aplicación se ha planteado siguiendo un procedimiento estructurado y dividido en fases, basadas en un **enfoque metodológico mixto**, que combina la investigación documental, el análisis comparativo y la experimentación técnica. El objetivo es garantizar que todas las decisiones de diseño y desarrollo estén fundamentadas en información real, buenas prácticas del sector y viabilidad tecnológica.

- **Fase 1: Análisis y definición de requisitos**

Se ha realizado un estudio exhaustivo del **sector del transporte de mercancías por carretera**, identificando oportunidades de optimización, necesidades de los usuarios y tendencias en plataformas digitales.

En esta fase, se han considerado dos perfiles principales:

- **Clientes que demandan el transporte de mercancías**, priorizando rapidez, seguridad, trazabilidad y facilidad de contratación.
- **Transportistas**, ya sean empresas con flotas infrautilizadas o autónomos con vehículos propios, con necesidades de eficiencia en la gestión de viajes, optimización de recursos y control de pagos.

La recopilación de información se ha llevado a cabo mediante análisis documental de informes del sector, revisión de plataformas existentes como OnTruck y estudio de soluciones tecnológicas y normativas vigentes.

- **Fase 2: Diseño conceptual y arquitectura**

El diseño de la aplicación se inspira en los principios de un **Transportation Management System (TMS)**, adaptado a un entorno simplificado y accesible desde dispositivos móviles. Este enfoque permite integrar funcionalidades clave de un TMS, como la planificación de viajes, gestión de flotas, trazabilidad de la mercancía y control de pagos, sin necesidad de desplegar infraestructura propia desde el inicio, aprovechando la escalabilidad de arquitecturas cloud.

- **Fase 3: Optimización de rutas**

Para garantizar la eficiencia en la asignación de viajes y la planificación de rutas, se considera la aplicación del **Travelling Salesman Problem (TSP)** como referencia teórica.

El TSP permite determinar la ruta óptima que minimiza la distancia total o el coste al visitar varios puntos de entrega, y aunque la versión inicial de la app no implementa algoritmos completos de TSP, este concepto sirve como guía para futuras mejoras en:

- Planificación de múltiples entregas por un mismo transportista.
- Minimización de tiempos de desplazamiento y costes operativos.
- Optimización de la flota y mejora del servicio ofrecido a clientes.

- **Fase 4: Diseño y desarrollo de la aplicación**

En esta fase se definirán:

- Interfaces de usuario centradas en la experiencia (UX) y facilidad de uso.
- Estructura de bases de datos para almacenar información de usuarios, vehículos, viajes y pagos.
- Integración de servicios cloud y pasarelas de pago para garantizar escalabilidad y seguridad.

El diseño se basa en la información recopilada en fases previas, asegurando que tanto clientes como transportistas puedan interactuar con la aplicación de manera ágil y confiable.

- **Fase 5: Pruebas y validación**

Se llevarán a cabo pruebas funcionales y conceptuales para:

- Verificar la correcta planificación de viajes y asignación de rutas.
- Evaluar la trazabilidad y seguridad de los envíos.
- Comprobar la correcta gestión de pagos y datos de usuarios.

Estas pruebas permitirán validar que la aplicación cumple con los objetivos definidos y que las decisiones tomadas en fases anteriores son efectivas.

6 DESARROLLO

Es la parte central del trabajo donde se van a implementar las ideas, teorías y metodologías previamente planteadas. Aquí se detallarán los pasos que se siguieron para alcanzar los objetivos del proyecto:

- Implementación: cómo se llevaron a cabo las actividades
- Resultados: presentación de datos
- Análisis: interpretación de esos resultados y relación con los objetivos planteados
- Discusión: evaluación crítica de los resultados comparándolos con estudios previos, posible repercusión y, en su caso, mejoras

6.1 Análisis del sistema

Los requisitos del sistema se han definido teniendo en cuenta los objetivos del proyecto, el perfil de los usuarios finales y el enfoque del desarrollo como un Producto Mínimo Viable (MVP). Este enfoque permite implementar un conjunto reducido de funcionalidades esenciales con el objetivo de validar la utilidad de la aplicación y analizar la respuesta inicial del mercado.

Asimismo, una parte de los requisitos ha sido seleccionada en base a los conocimientos y tecnologías adquiridos durante el presente curso académico, con el propósito de aplicar de forma práctica los contenidos vistos en clase. El autor del proyecto considera que los lenguajes y herramientas empleadas durante su formación constituyen una opción lo suficientemente robusta y adecuada para el desarrollo del proyecto, permitiendo cumplir los objetivos planteados sin recurrir a soluciones excesivamente complejas.

En cuanto a la compatibilidad, aunque la aplicación se concibe para poder ser utilizada tanto en dispositivos Android como iOS, al tratarse de una primera versión orientada a la validación del concepto, se ha decidido desarrollar inicialmente la aplicación únicamente para dispositivos Android. Esta decisión permite reducir la complejidad técnica del proyecto y centrar los esfuerzos en la funcionalidad principal del sistema.

Con el fin de estructurar correctamente el análisis, los requisitos se han dividido en requisitos funcionales y requisitos no funcionales, describiendo tanto las funcionalidades que debe ofrecer el sistema como las condiciones bajo las cuales debe operar.

6.1.1 Especificación de requisitos

La especificación de requisitos recoge de forma estructurada las funcionalidades que el sistema debe ofrecer, así como las restricciones y condiciones de calidad que debe cumplir. Para ello se distingue entre requisitos funcionales (qué hace el sistema) y requisitos no funcionales (cómo debe comportarse el sistema).

Requisitos funcionales

Los requisitos funcionales describen las operaciones concretas que el sistema debe ser capaz de realizar. Para su priorización se ha utilizado la escala Alta / Media / Baja, donde Alta indica que la funcionalidad es imprescindible para el MVP, Media que aporta valor pero puede posponerse sin invalidar el producto, y Baja que se incluirá en fases posteriores.

Tabla 1 Requisitos funcionales del sistema

ID	Nombre	Descripción	Prioridad	Actor
RF-01	Registro de usuario	El sistema debe permitir registrar nuevos usuarios indicando nombre, apellidos, email, contraseña y rol (CLIENTE o TRANSPORTISTA).	Alta	Ambos

ID	Nombre	Descripción	Prioridad	Actor
RF-02	Inicio de sesión	El sistema debe autenticar al usuario mediante email y contraseña, devolviendo un token JWT que se usará en el resto de operaciones.	Alta	Ambos
RF-03	Gestión de vehículos	El transportista puede dar de alta vehículos asociados a su cuenta, indicando matrícula, marca, modelo, tipo de vehículo, capacidad en kg y tipos de mercancía admitidos.	Alta	Transportista
RF-04	Crear solicitud de viaje	El cliente puede crear una nueva solicitud de transporte especificando dirección de recogida, dirección de entrega, tipo de mercancía, peso y descripción opcional.	Alta	Cliente
RF-05	Consultar viajes disponibles	El transportista puede consultar las solicitudes de viaje en estado PENDIENTE que sean compatibles con los tipos de mercancía de sus vehículos.	Alta	Transportista
RF-06	Aceptar viaje	El transportista puede aceptar un viaje disponible, quedando asociado a dicho viaje junto con el vehículo seleccionado. El estado pasa a ACEPTADO.	Alta	Transportista
RF-07	Actualizar estado del viaje	El transportista puede actualizar el estado del viaje siguiendo la secuencia definida: SALIDA_RECOGIDA → LLEGADA_RECOGIDA → MERCANCIA_RECOGIDA → SALIDA_ENTREGA → LLEGADA_ENTREGA → COMPLETADO.	Alta	Transportista
RF-08	Consultar estado del viaje	El cliente puede consultar en cualquier momento el estado actual de sus viajes en curso e históricos.	Alta	Cliente
RF-09	Historial de viajes	Ambos actores pueden acceder al historial de viajes (completados y cancelados) asociados a su cuenta.	Media	Ambos
RF-10	Cancelar viaje	Un viaje puede ser cancelado desde cualquier estado previo a COMPLETADO. El sistema registra la cancelación y libera el vehículo asociado si lo hubiera.	Media	Ambos
RF-11	Compatibilidad vehículo-mercancía	El sistema debe validar automáticamente que el vehículo del transportista puede transportar el tipo de mercancía indicado en el viaje antes de permitir la aceptación.	Alta	Sistema

ID	Nombre	Descripción	Prioridad	Actor
RF-12	Cierre de sesión	El usuario puede cerrar sesión desde la aplicación, eliminando el token JWT almacenado localmente.	Baja	Ambos

Requisitos no funcionales

Los requisitos no funcionales establecen las cualidades que debe tener el sistema en términos de seguridad, rendimiento, usabilidad, escalabilidad y cumplimiento normativo. Aunque no describen funcionalidades concretas, condicionan las decisiones de diseño y arquitectura del proyecto.

Tabla 2 Requisitos no funcionales del sistema

ID	Nombre	Descripción	Categoría
RNF-01	Seguridad	Todas las contraseñas deben almacenarse cifradas mediante BCrypt. La autenticación se realizará con tokens JWT con tiempo de expiración configurado. Todas las rutas, excepto registro y login, requieren token válido.	Seguridad
RNF-02	Rendimiento	La API debe responder a las peticiones habituales (consulta de viajes, actualización de estado) en menos de 2 segundos bajo condiciones de carga normal.	Rendimiento
RNF-03	Usabilidad	La interfaz de la aplicación debe ser intuitiva y accesible para usuarios sin experiencia técnica. Los flujos principales (crear viaje, aceptar viaje) deben completarse en un máximo de 3 pasos.	Usabilidad
RNF-04	Compatibilidad	La aplicación Android debe ser compatible con dispositivos con Android 8.0 (API 26) o superior, cubriendo la mayor parte del parque de dispositivos activo.	Compatibilidad
RNF-05	Escalabilidad	La arquitectura backend basada en Spring Boot y MySQL debe permitir escalar horizontalmente en el futuro sin cambios estructurales. La lógica de negocio estará desacoplada de la infraestructura.	Arquitectura
RNF-06	Mantenibilidad	El código debe seguir convenciones de nomenclatura consistentes, separación en capas (Controller, Service,	Mantenibilidad

ID	Nombre	Descripción	Categoría
		Repository) y estar documentado en las partes críticas para facilitar el mantenimiento y la evolución futura.	
RNF-07	Disponibilidad	La API debe estar disponible durante las pruebas funcionales sin interrupciones. En fases futuras de producción se contemplará un SLA mínimo del 99,5%.	Disponibilidad
RNF-08	Privacidad de datos	Los datos personales de usuarios (email, teléfono) no se expondrán en respuestas públicas ni se compartirán entre actores sin consentimiento explícito, de acuerdo al RGPD.	Legal

Casos de uso

Los casos de uso representan las interacciones entre los actores del sistema y las funcionalidades ofrecidas por la aplicación. Se han identificado dos actores principales: el Cliente, que solicita servicios de transporte, y el Transportista, que los ejecuta. A continuación se presenta la descripción resumida de los casos de uso principales:

Tabla 3.Casos de uso principales.

ID	Caso de uso	Actor	Precondición	Flujo principal
CU-01	Registrarse	Nuevo usuario	Sin sesión iniciada	El usuario introduce sus datos (nombre, email, contraseña, rol) y el sistema crea la cuenta, devolviendo confirmación.
CU-02	Iniciar sesión	Usuario registrado	Cuenta existente	El usuario introduce email y contraseña; el sistema valida las credenciales y devuelve un JWT.
CU-03	Crear viaje	Cliente	Sesión iniciada como CLIENTE	El cliente rellena origen, destino, tipo de mercancía y peso. El sistema crea el viaje en estado PENDIENTE.
CU-04	Consultar viajes disponibles	Transportista	Sesión iniciada como TRANSPORTISTA, vehículo registrado	El sistema filtra y muestra los viajes PENDIENTES compatibles con los vehículos del transportista.

ID	Caso de uso	Actor	Precondición	Flujo principal
CU-05	Aceptar viaje	Transportista	Viaje en estado PENDIENTE, transportista con vehículo compatible	El transportista selecciona un viaje y un vehículo; el sistema cambia el estado a ACEPTADO y asocia al transportista.
CU-06	Actualizar estado	Transportista	Viaje asignado al transportista en curso	El transportista avanza el estado del viaje al siguiente paso de la secuencia definida.
CU-07	Consultar estado	Cliente	Sesión iniciada como CLIENTE	El cliente accede al detalle de un viaje y visualiza el estado actual y las marcas de tiempo de cada transición.
CU-08	Cancelar viaje	Ambos	Viaje en estado distinto a COMPLETADO	El actor solicita la cancelación; el sistema cambia el estado a CANCELADO y libera el vehículo si estaba asignado.
CU-09	Gestionar vehículos	Transportista	Sesión iniciada como TRANSPORTISTA	El transportista da de alta, consulta o elimina vehículos asociados a su cuenta, indicando los tipos de mercancía admitidos.

El diagrama de casos de uso (Figura X) muestra de forma gráfica las relaciones entre actores y funcionalidades. Ambos actores comparten los casos de uso de registro, inicio de sesión, consulta de historial y cancelación de viaje, mientras que los flujos de creación y seguimiento de viajes son exclusivos del Cliente y los de aceptación y actualización de estado son exclusivos del Transportista.

6.2 Tecnologías utilizadas

En este apartado se describen y justifican las tecnologías seleccionadas para el desarrollo del proyecto PISPAX. La elección del stack tecnológico se ha basado en tres criterios principales: la adecuación al perfil formativo del ciclo DAM, la viabilidad técnica dentro del alcance de un MVP, y la coherencia con las arquitecturas usadas en el sector real del transporte digital

6.2.1 Aplicación Android (Frontend)

La capa cliente de la aplicación se ha desarrollado como una aplicación nativa Android. Esta decisión responde a varios factores: el lenguaje Java es el principal lenguaje trabajado durante la formación, el entorno Android Studio ofrece todas las herramientas necesarias para el desarrollo, depuración y prueba en emulador, y la plataforma Android concentra la mayor parte del mercado móvil en España, lo que la convierte en la opción más adecuada para validar el MVP. Se descartó el desarrollo multiplataforma con Flutter o React Native por el sobrecoste de aprendizaje que implicaría en el contexto de este proyecto.

Tabla 4.Stack

Componente	Tecnología	Versión	Justificación
Lenguaje	Java	17	Lengua principal del ciclo DAM. Compatible con Android Studio y Spring Boot.
IDE	Android Studio	Ladybug	Entorno oficial para desarrollo Android. Incluye emulador y depurador integrado.
Interfaz de usuario	XML Layouts nativos	Nativo	Permite diseñar pantallas de forma visual y declarativa sin librerías externas.
Componentes UI	Material Design	1.11.0	Biblioteca oficial de Google. Botones, campos, tarjetas y navegación con estética moderna.
Cliente HTTP	Retrofit	2.9.0	Librería estándar para consumir APIs REST en Android. Simplifica peticiones HTTP y deserialización.
Serialización JSON	Gson (via Retrofit)	2.9.0	Conversor JSON↔POJO integrado con Retrofit. Manejo transparente de las respuestas de la API.
Listas dinámicas	RecyclerView	1.3.2	Componente eficiente para mostrar listas de viajes y vehículos con reciclaje de vistas.
Sesión local	SharedPreferences	Nativo	Almacenamiento ligero para persistir el token JWT entre sesiones sin base de datos local.

6.2.2 API REST (Backend)

El backend se implementa como una API REST stateless desarrollada con Spring Boot. Esta arquitectura desacoplada permite que la aplicación Android y cualquier cliente futuro (versión iOS, panel web) consuman los mismos servicios sin duplicar lógica de negocio. Spring Boot fue elegido por ser el framework Java más extendido en el mercado laboral, por su ecosistema maduro de librerías y por la rapidez con la que permite montar una API funcional con seguridad, validación y gestión de base de datos integradas.

Tabla 5.Stack Back

Componente	Tecnología	Versión	Justificación
Lenguaje	Java	17	Consistencia con el cliente Android. Versión LTS con soporte a largo plazo.
Framework	Spring Boot	3.2.x	Configuración automática, servidor embebido Tomcat y amplio ecosistema de starters.
ORM	Spring Data JPA + Hibernate	Boot 3.2	Abstrae el acceso a base de datos. Permite definir consultas con anotaciones sin SQL manual en la mayoría de casos.
Seguridad	Spring Security + JWT (jjwt)	0.12.x	Autenticación sin estado mediante tokens. Protege todos los endpoints excepto /api/auth/*.

Reducción de boilerplate	Lombok	1.18.x	Genera getters, setters, constructores y builders en tiempo de compilación. Reduce el tamaño del código.
Validación	spring-boot-starter-validation	3.2.x	Valida automáticamente los DTO de entrada mediante anotaciones (@NotNull, @Email, etc.).
Gestión de dependencias	Maven	3.9	Herramienta estándar del sector. Facilita la gestión del ciclo de vida del proyecto.

6.2.3 Base De Datos

Para la capa de persistencia se utiliza MySQL 8, un sistema de gestión de bases de datos relacional ampliamente adoptado en el sector empresarial. La naturaleza relacional de los datos del proyecto (usuarios, vehículos, tipos de mercancía, viajes) encaja perfectamente con el modelo tabular de SQL, donde las relaciones entre entidades se expresan mediante claves foráneas. Frente a alternativas NoSQL como MongoDB, el esquema relacional aporta mayor integridad referencial, algo crítico en un sistema donde un viaje debe estar siempre vinculado a un cliente y, una vez aceptado, a un transportista y un vehículo.

Tabla 6.DB

Componente	Tecnología	Justificación
SGBD	MySQL 8	Relacional, maduro y ampliamente compatible con Spring Data JPA. Alta disponibilidad de documentación y soporte.
Driver JDBC	mysql-connector-j 8.x	Driver oficial de MySQL para Java. Integrado automáticamente por Spring Boot al incluir la dependencia.
Gestión del esquema	Hibernate DDL auto-update	En fase de desarrollo permite actualizar el esquema automáticamente al iniciar la aplicación, agilizando la iteración.

6.2.4 Comunicación Cliente-Servidor

La comunicación entre la app Android y el backend se realiza a través de HTTP/HTTPS con intercambio de datos en formato JSON. El esquema de autenticación elegido es JWT (JSON Web Token): al iniciar sesión, el servidor devuelve un token firmado que el cliente almacena en SharedPreferences y adjunta en la cabecera Authorization de todas las peticiones posteriores. Este enfoque stateless elimina la necesidad de mantener sesiones en el servidor y facilita el escalado futuro de la API.

6.2.5 Resumen y justificación del stack

La siguiente tabla resume las tecnologías clave del proyecto y los criterios que motivaron su selección frente a alternativas:

Tabla 7.Justificacion del Stack

Tecnología	Alternativa considerada	Razón de elección
------------	-------------------------	-------------------

Java 17	Kotlin	Lenguaje principal del ciclo formativo. Curva de aprendizaje ya superada.
Spring Boot	Node.js / Express	Ecosystem Java robusto, mayor integración con JPA/Hibernate y Spring Security.
MySQL	PostgreSQL / MongoDB	Mayor familiaridad. Soporte nativo en Spring Boot sin configuración adicional.
Retrofit + Gson	OkHttp manual / Ktor	Abstracción de alto nivel para APIs REST. Estándar en proyectos Android Java.
JWT	OAuth2 / Firebase Auth	Implementación directa sin servicios externos. Suficiente para el alcance del MVP.
XML Layouts	Jetpack Compose	Compatible con Android Studio sin requisitos adicionales. Menor curva para el MVP.

6.3 Diseño del sistema

6.3.1 Arquitectura de la aplicación

La arquitectura de PisPax sigue el patrón cliente-servidor, donde la aplicación Android actúa como cliente y el servidor Spring Boot expone una API REST stateless. La comunicación entre ambas capas se realiza mediante HTTP con intercambio de datos en formato JSON y autenticación basada en tokens JWT.

Esquema general:

[App Android] —HTTP/JSON—> [API REST Spring Boot] —JPA/SQL—> [MySQL 8]

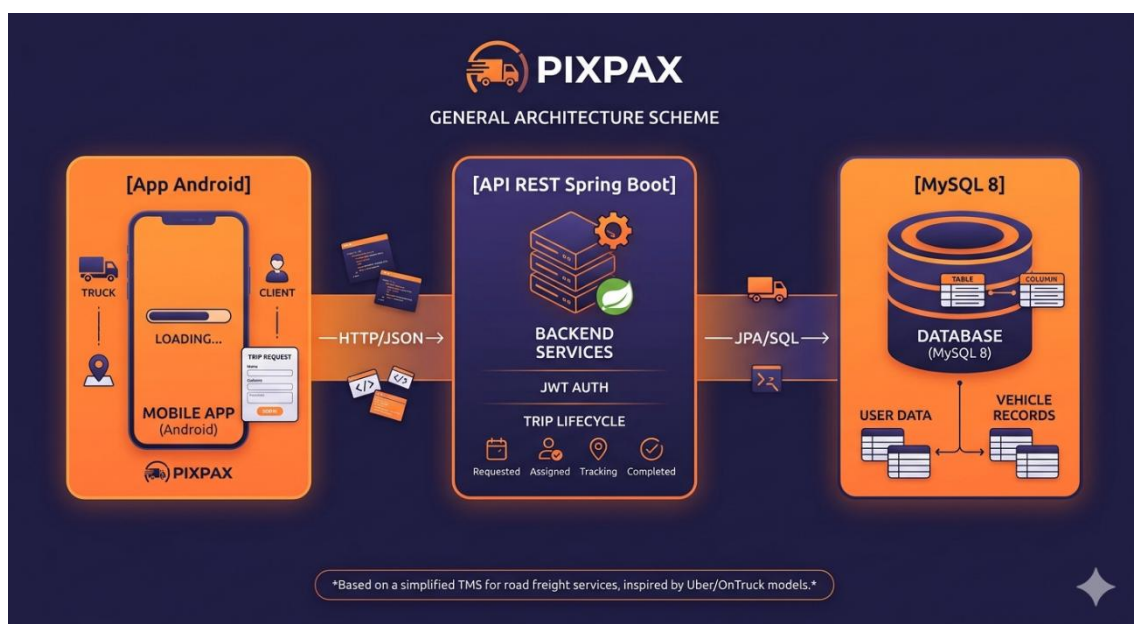


Ilustración 1. Esquema

El backend se estructura en tres capas bien diferenciadas:

- **Controller:** Recibe las peticiones HTTP, valida la entrada y delega la lógica en la capa Service. Devuelve ResponseEntity con el código HTTP semántico correspondiente.
- **Service:** Contiene toda la lógica de negocio. Es responsable de las validaciones de estado, la compatibilidad vehículo-mercancía y la coordinación entre repositorios.
- **Repository:** Interfaz JpaRepository que gestiona el acceso a datos. Únicamente contiene consultas personalizadas cuando las derivadas automáticas no son suficientes.

Los DTOs (Data Transfer Objects) actúan como objetos de transferencia entre capas, garantizando que nunca se expone directamente la entidad JPA al exterior de la API.

Seguridad JWT:

El flujo de autenticación funciona de la siguiente manera: el cliente realiza una petición POST a /api/auth/login con email y contraseña. El servidor valida las credenciales, genera un token JWT firmado y lo devuelve al cliente. A partir de ese momento, el cliente incluye el token en la cabecera Authorization: Bearer {token} de cada petición. El filtro JwtAuthFilter intercepta y valida el token antes de que la petición llegue al controller, devolviendo 401 Unauthorized si el token es inválido o ha expirado.

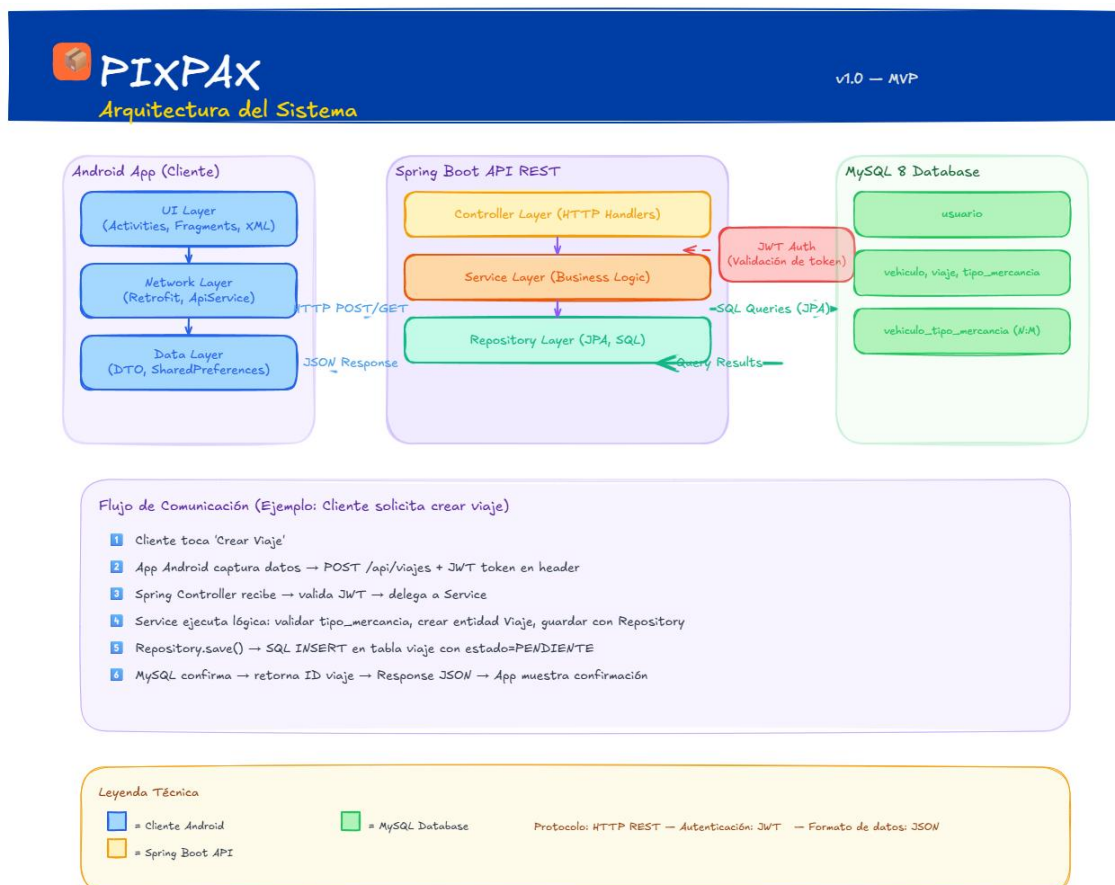


Ilustración 2.Arquitectura

6.3.2 Diseño de la base de datos

La base de datos de PisPax está diseñada sobre MySQL 8 siguiendo el modelo relacional. Contiene 5 tablas principales que cubren las entidades del dominio: **usuarios** (clientes y transportistas), **vehículos**, **tipos de mercancía**, la **relación de compatibilidad** entre vehículos y mercancías, y los **viajes**.

Se utilizan ENUMs nativos de MySQL para modelar los estados finitos del sistema, lo que garantiza integridad a nivel de base de datos. El catálogo de tipos de vehículo incluye 7 categorías y el de tipos de mercancía 12 categorías, diseñados para reflejar la realidad del mercado español de transporte de mercancías por carretera.

Tabla: usuario

Tabla central del sistema. Almacena tanto clientes como transportistas, diferenciados por el campo rol. Un único registro por persona evita duplicidades y simplifica la gestión de sesiones.

Tabla 8.Tabla Usuario

Campo	Tipo	Restricción	Descripción
id	BIGINT	PK, AUTO_INCREMENT	Identificador único del usuario
nombre	VARCHAR(100)	NOT NULL	Nombre del usuario
apellidos	VARCHAR(150)	NOT NULL	Apellidos del usuario
email	VARCHAR(200)	NOT NULL, UNIQUE	Correo electrónico — usado como login
password_hash	VARCHAR(255)	NOT NULL	Contraseña cifrada con BCrypt
telefono	VARCHAR(20)	NULL	Teléfono de contacto
rol	ENUM	NOT NULL	CLIENTE o TRANSPORTISTA
created_at	TIMESTAMP	DEFAULT NOW()	Fecha de registro

Tabla: vehiculo

Registra los vehículos que los transportistas dan de alta en la plataforma. Incluye campos para la validación técnica y legal: subtipo de vehículo, tara y masa máxima autorizada (MMA), y tipo de carnet requerido según la normativa española de transporte de mercancías.

Tabla 9.Tabla Vehiculo

Campo	Tipo	Restricción	Descripción
id	BIGINT	PK, AUTO_INCREMENT	Identificador único del vehículo
matricula	VARCHAR(15)	NOT NULL, UNIQUE	Matrícula del vehículo
marca	VARCHAR(80)	NOT NULL	Marca del vehículo
modelo	VARCHAR(80)	NOT NULL	Modelo del vehículo
tipo_vehiculo	ENUM	NOT NULL	Categoría principal del vehículo
subtipo	VARCHAR(50)	NULL	Subclasificación (Compacta, Mediana, Volumen alto)
tara_kg	DECIMAL(8,2)	NULL	Peso en vacío del vehículo (kg)
capacidad_kg	DECIMAL(8,2)	NOT NULL	Carga útil máxima en kilogramos
mma_kg	DECIMAL(8,2)	NULL	Masa Máxima Autorizada (kg)
carnet_requerido	ENUM	NOT NULL DEFAULT 'B'	Carnet de conducir necesario: B, C1, C, C+E
transportista_id	BIGINT	FK → usuario.id	Propietario del vehículo

Nota: Los campos tara_kg y mma_kg permiten al sistema validar que el peso de la mercancía solicitada no supere la carga útil del vehículo. El campo carnet_requerido es clave para la validación del perfil del transportista en el proceso de alta.

Tabla: tipo_mercancia

Catálogo de categorías de mercancía que el sistema reconoce. Contiene 12 tipos de mercancía pre-cargados al arrancar la aplicación. Se añade el campo nombre_db con el identificador snake_case utilizado internamente por la API.

Tabla 10. Tipo Mercancia

Campo	Tipo	Restricción	Descripción
id	BIGINT	PK, AUTO_INCREMENT	Identificador del tipo de mercancía
nombre	VARCHAR(100)	NOT NULL, UNIQUE	Nombre de la categoría (legible)
nombre_db	VARCHAR(100)	NOT NULL, UNIQUE	Clave snake_case para uso interno en la API
descripcion	VARCHAR(500)	NULL	Descripción informativa del tipo

Tabla: vehiculo_tipo_mercancia

Tabla de unión N:M entre vehículos y tipos de mercancía. Incluye el campo compatible (ENUM: SI / CON_REQUISITOS) para registrar compatibilidades condicionales. Por ejemplo, un camión C1 puede transportar alimentos frescos solo si dispone de módulo frigorífico homologado.

Tabla 11. Mercancia-Vehículo

Campo	Tipo / Restricción / Descripción
vehiculo_id	BIGINT — PK, FK → vehiculo.id — Referencia al vehículo
tipo_mercancia_id	BIGINT — PK, FK → tipo_mercancia.id — Referencia al tipo de mercancía
compatible	ENUM — NOT NULL DEFAULT 'SI' — Valores: SI CON_REQUISITOS

Nota: La compatibilidad CON_REQUISITOS implica que la lógica de negocio (capa Service) debe verificar los requisitos adicionales (ADR, frío, GDP...) antes de asignar el viaje al transportista.

Tabla: viaje

Entidad central de la operativa. Registra cada solicitud de transporte desde su creación hasta su finalización. Los campos transportista_id y vehiculo_id son NULL en el momento de creación (estado PENDIENTE) y se rellenan cuando un transportista acepta el viaje.

Tabla 12. Viaje

Campo	Tipo	Restricción	Descripción
id	BIGINT	PK, AUTO_INCREMENT	Identificador del viaje
cliente_id	BIGINT	FK → usuario.id, NOT NULL	Cliente que solicita el servicio
transportista_id	BIGINT	FK → usuario.id, NULL	Transportista asignado (tras aceptación)
vehiculo_id	BIGINT	FK → vehiculo.id, NULL	Vehículo utilizado (tras aceptación)
tipo_mercancia_id	BIGINT	FK → tipo_mercancia.id	Categoría de mercancía del envío
descripcion_mercancia	VARCHAR(500)	NULL	Descripción libre de la carga
direccion_recogida	VARCHAR(300)	NOT NULL	Punto de origen del envío
direccion_entrega	VARCHAR(300)	NOT NULL	Punto de destino del envío
peso_kg	DECIMAL(8,2)	NOT NULL	Peso estimado de la mercancía
estado	ENUM	NOT NULL, DEFAULT PENDIENTE	Estado actual del viaje
fecha_solicitud	TIMESTAMP	DEFAULT NOW()	Momento de creación del viaje
fecha_inicio	TIMESTAMP	NULL	Momento en que el transportista sale a recoger
fecha_fin	TIMESTAMP	NULL	Momento de entrega completada

ENUMs definidos

ENUM: rol (tabla usuario)

Tabla 13. Rol Usuarios

Valor	Descripción
CLIENTE	Usuario que solicita servicios de transporte
TRANSPORTISTA	Usuario que ofrece y ejecuta servicios de transporte

ENUM: tipo_vehiculo (tabla vehiculo)

Tabla 14. Tipo Vehiculos

Valor	Descripción	MMA máx. (kg)	Carnet requerido
FURGONETA	Vehículo ligero de reparto. Compacta o mediana.	Hasta 3.500	B
FURGON_GRANDE	Furgón de alto volumen. Ideal para mobiliario.	Hasta 3.500	B
FRIGORIFICO	Vehículo con cámara de temperatura controlada.	Hasta 3.500	B
CAMION_LIGERO	Camión rígido pequeño para distribución regional.	3.500 – 7.500	C1
CAMION_PESADO	Camión rígido de gran tonelaje. Requiere CAP.	> 7.500	C + CAP
CAMION_ARTICULADO	Semirremolque para larga distancia. Requiere CAP.	Hasta 42.000	C+E + CAP
PLATAFORMA	Vehículo para cargas voluminosas, palets o maquinaria.	Variable	C1/C

ENUM: carnet_requerido (tabla vehiculo)

Tabla 15.Carnet Requerido

Valor	Rango MMA	Descripción
B	Hasta 3.500 kg MMA	Carnet estándar. Válido para furgonetas y furgones ligeros.
C1	3.500 – 7.500 kg MMA	Requiere formación adicional. Para camiones ligeros de distribución regional.
C	Más de 7.500 kg MMA	Requiere CAP (Certificado de Aptitud Profesional). Para camiones pesados.
C+E	Articulado o con remolque > 750 kg	Requiere C previo y CAP. Para semirremolques y transporte de larga distancia.

ENUM: estado (tabla viaje)

Tabla 16.Estado Viaje

Valor	Descripción
PENDIENTE	Viaje creado, esperando que un transportista lo acepte.
ACEPTADO	Transportista asignado, aún no ha iniciado el desplazamiento.
SALIDA_RECOGIDA	El transportista ha salido hacia el punto de recogida.
LLEGADA_RECOGIDA	El transportista ha llegado al punto de recogida.
MERCANCIA_RECOGIDA	La mercancía ha sido cargada en el vehículo.
SALIDA_ENTREGA	El transportista ha salido hacia el punto de entrega.
LLEGADA_ENTREGA	El transportista ha llegado al punto de entrega.
COMPLETADO	Mercancía descargada y entrega finalizada satisfactoriamente.
CANCELADO	Viaje cancelado por cliente o transportista (accesible desde cualquier estado).

Relaciones entre tablas

Tabla 17.Relacion de Tablas

Tabla origen	Tabla destino	Cardinalidad	Descripción
usuario	vehiculo	1 : N	Un transportista puede tener varios vehículos registrados
usuario (cliente)	viaje	1 : N	Un cliente puede tener varios viajes solicitados
usuario (transportista)	viaje	1 : N	Un transportista puede gestionar varios viajes
vehiculo	viaje	1 : N	Un vehículo puede ser asignado a varios viajes distintos
vehiculo	vehiculo_tipo_mercancia	1 : N	Un vehículo acepta varios tipos de mercancía
tipo_mercancia	vehiculo_tipo_mercancia	1 : N	Un tipo puede ser aceptado por varios vehículos
tipo_mercancia	viaje	1 : N	Un tipo de mercancía puede aparecer en varios viajes

Datos semilla — tipo_mercancia (12 registros)

Los siguientes registros se insertan automáticamente al iniciar la aplicación. El campo nombre_db es el identificador utilizado internamente por la API REST para filtrar compatibilidades.

Tabla 18.Datos Semillas para MVP

ID	Nombre	nombre_db	Vehículo recomendado	Doc. especial
1	Paquetería general	paqueteria_general	Furgoneta / Furgón	No
2	Electrónica frágil	electronica_fragil	Furgón con protección	No

3	Alimentos frescos	alimentos_frescos	Frigorífico (0°C–8°C)	No
4	Alimentos congelados	alimentos_congelados	Frigorífico (< -18°C)	No
5	Materiales construcción	materiales_construccion	Camión / Furgón carga pesada	No
6	Mobiliario	mobiliario	Furgón grande / Camión	No
7	Mercancía peligrosa	mercancia_peligrosa	Vehículo habilitado ADR	Sí — ADR
8	Productos farmacéuticos	productos_farmaceuticos	Furgón temp. controlada	GDP (recom.)
9	Textil y moda	textil_moda	Furgoneta / Furgón	No
10	Maquinaria industrial	maquinaria_industrial	Camión plataforma	No
11	Documentación y archivo	documentacion_archivo	Furgoneta estándar	No
12	Vehículos y automoción	vehiculos_automocion	Camión plataforma / portavehículos	No

Decisiones de diseño

- Tabla única de usuarios con campo rol: evita duplicidades, simplifica el login y permite que un usuario pueda cambiar de rol en el futuro sin migración de datos.
- transportista_id y vehiculo_id nullables en viaje: el viaje existe desde que el cliente lo crea, antes de que haya transportista disponible. Los NULL son semánticamente correctos en este contexto.
- ENUMs en base de datos: garantizan integridad a nivel de motor, sin depender exclusivamente de la validación en la capa de aplicación.
- Tabla vehiculo_tipo_mercancia con campo compatible: permite distinguir compatibilidades absolutas de condicionales, facilitando la implementación de validaciones adicionales en la capa Service sin alterar el esquema.
- Campos tara_kg y mma_kg en vehiculo: permiten calcular la carga útil real disponible y validar que el peso del viaje no supera la capacidad del vehículo.
- nombre_db en tipo_mercancia: desacopla el nombre de presentación del identificador técnico, facilitando cambios de naming sin afectar a la lógica de la API.

Aquí queda Meter Diagrama de BD, Algun diagrama mas aunque ya esta el Diagrama de caso de uso UML -> el propósito de este es mostrar que puede hacer el sistema y quien lo usa.

Estaría bastante bien crear:

Diagrama de Secuencia Muestra cómo se comunican los componentes en el tiempo para completar una operación concreta. Por ejemplo: el flujo completo de "crear un viaje" desde que el cliente pulsa el botón hasta que la base de datos guarda el registro. Responde a: *¿en qué orden ocurren las cosas?*

Diagrama de Actividad Es como un diagrama de flujo, pero con notación UML. Ideal para representar los 6+2 estados del viaje y las transiciones entre ellos. Responde a: *¿qué camino sigue un proceso?*

Diagrama de Estados (State Machine) Específico para modelar los estados de un objeto y sus transiciones. Es perfectamente adecuado para el ENUM de estados del viaje: PENDIENTE → ACEPTADO → SALIDA_RECOGIDA → ... → COMPLETADO. Responde a: *¿cómo cambia el estado de una entidad?*

Diagramas estructurales (cómo está construido)

Diagrama de Clases El más importante para el backend. Muestra las clases Java (Usuario, Viaje, Vehículo...), sus atributos, métodos y relaciones (herencia, asociación, composición). Responde a: *¿cómo está organizado el código?*

Diagrama Entidad-Relación (E/R) No es UML, pero es estándar en bases de datos. Muestra las tablas, sus campos y las relaciones entre ellas. Es lo que necesitas para la sección **4.3.2 Diseño de la base de datos**. Responde a: *¿cómo está estructurada la base de datos?*

Diagrama de Componentes / Arquitectura Muestra los bloques grandes del sistema y cómo se conectan: App Android → API REST → MySQL. Es lo que necesitas para **4.3.1 Arquitectura de la aplicación**. Responde a: *¿qué partes tiene el sistema y cómo se comunican?*

PIXPAX — Tabla de Compatibilidad: Tipo de Vehículo vs. Tipo de Mercancía							
☑ = Compatible ✗ = No compatible ⚠ = Compatible con requisitos adicionales							
Tipo de Mercancía ↓ / Vehículo →	Furgoneta	Furgón Grande	Frigorífico	Camión Ligero C1	Camión Pesado C	Camión Articulado C+E	Plataforma
Paquetería general	☑	☑	☑	☑	☑	☑	✗
Electrónica frágil	☑	☑	✗	☑	☑	☑	✗
Alimentos frescos	✗	✗	☑	⚠	⚠	⚠	✗
Alimentos congelados	✗	✗	☑	⚠	⚠	⚠	✗
Mat. construcción	✗	⚠	✗	☑	☑	☑	☑
Mobiliario	✗	☑	✗	☑	☑	☑	⚠
Mercancía peligrosa	⚠	⚠	⚠	⚠	⚠	⚠	✗
Prod. farmacéuticos	✗	⚠	☑	⚠	⚠	⚠	✗
Textil y moda	☑	☑	✗	☑	☑	☑	✗
Maquinaria industrial	✗	✗	✗	⚠	☑	☑	☑
Documentación	☑	☑	✗	☑	☑	☑	✗
Vehículos / Automoción	✗	✗	✗	✗	⚠	⚠	☑

Ilustración 3. Tabla de compatibilidad

6.3.3 Diseño de la interfaz de usuario (UI)

La aplicación utiliza una paleta de colores diseñada para: - Garantizar contraste y accesibilidad (WCAG AA mínimo) - Facilitar diferenciación de roles (cliente vs. transportista) - Transmitir confianza y profesionalidad en el sector logístico.

PIXPAX — Combinaciones de color recomendadas				
Combinación	Fondo	Texto / Acento	Ratio WCAG	Uso
CTA principal	#FF8E3B	#FFFFFF	3.1:1 (AA UI)	Botón primario, botón Aceptar viaje
CTA hover / pressed	#E06B15	#FFFFFF	4.2:1 (AA)	Estado hover/pressed del botón primario
Navbar / Header	#2D2850	#FF8E3B	5.4:1 (AA+)	Barra de navegación, cabeceras de sección
Fondo de pantalla + texto	#FFFFFF	#1A1535	14.7:1 (AAA)	Pantallas principales, texto corrido
Card estándar	#FAF9F8	#1A1535	13.8:1 (AAA)	Tarjetas de viaje, lista de vehículos
Badge naranja	#FF8E3B	#6B2E00	4.1:1 (AA)	Badges de estado, etiquetas de precio
Badge navy	#2D2850	#FFFFFF	12.6:1 (AAA)	Etiquetas de rol, chip de transportista
Input activo	#FFFFFF	#FF8E3B	3.1:1	Borde de campo de texto activo
Dark mode — fondo	#0D0B1A	#FF8E3B	8.2:1 (AAA)	Fondo pantalla modo oscuro
Dark mode — card	#1A1535	#FAF9F8	13.1:1 (AAA)	Cards en modo oscuro
Éxito	#E8F7EE	#1A6B3F	5.9:1 (AA+)	Toast viaje completado, pago exitoso
Error	#FEE8E8	#8B0000	6.3:1 (AA+)	Mensajes de error, cancelación
Alerta	#FFF8E7	#8A5A00	5.1:1 (AA+)	Viaje pendiente de aceptación
Info	#E8F0FE	#1A3A8B	6.7:1 (AA+)	Tooltips, información de ayuda

Ilustración 4. Paleta de Colores

6.4 Implementación

La implementación de PisPax ha seguido un enfoque iterativo basado en fases estructuradas, donde cada fase construye sobre los resultados de la anterior, garantizando que las decisiones arquitectónicas y de diseño estén validadas antes de avanzar. Este flujo de trabajo ha sido diseñado para maximizar la viabilidad técnica del MVP dentro de los plazos disponibles y permitir evolución futura sin necesidad de refactorización mayor.

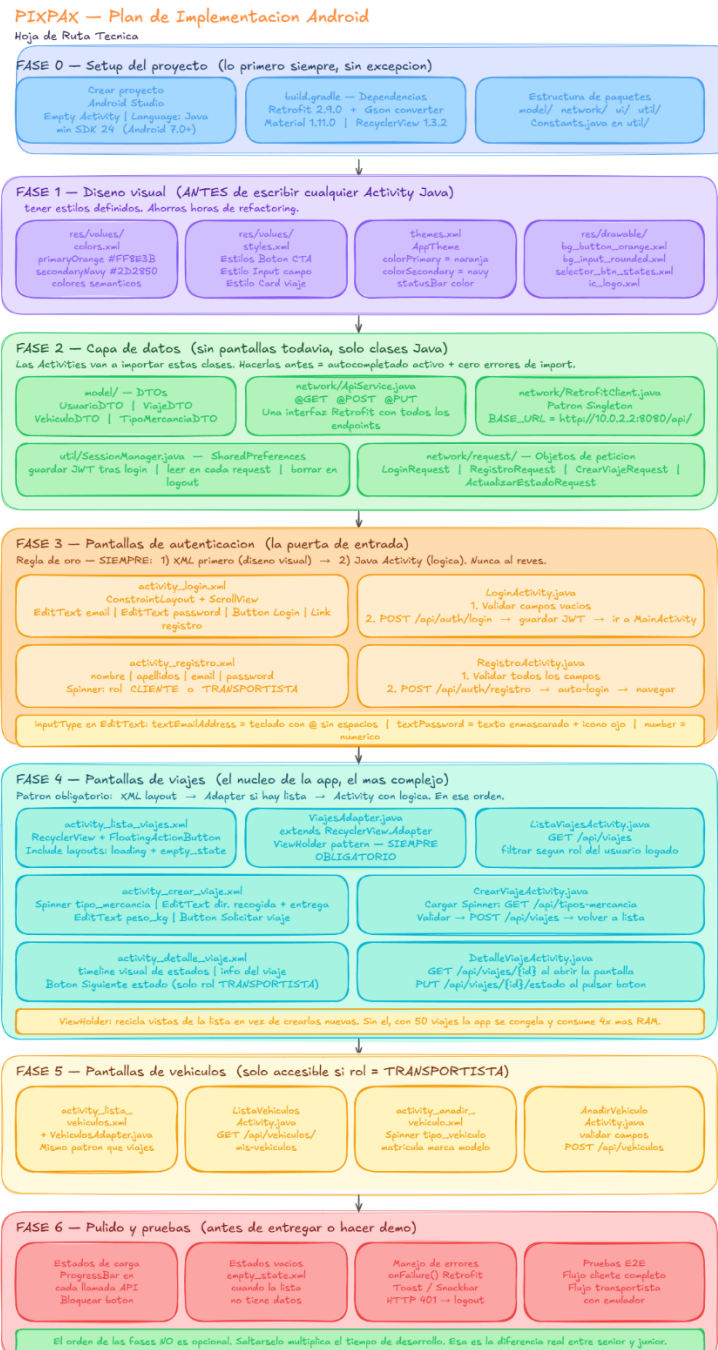


Ilustración 5. Workflow

4.5 Pruebas realizadas

7 ANÁLISIS

Interpretación de esos resultados y relación con los objetivos planteados.

7.1 Análisis del cumplimiento de objetivos

7.2 Evaluación del sistema desarrollado

7.3 Comparación con soluciones existentes

7.4 Limitaciones del proyecto

8 CONCLUSIONES

Discusión: evaluación crítica de los resultados comparándolos con estudios previos, posible repercusión, valoración personal y profesional, posibles líneas de mejora o continuidad.

9 Manual Para Usuario / Documentación Caso Practico

Manual Pendiente de Subir las Imágenes, hasta que no se este al 100% no se subirá este apartado del Documento

10 BIBLIOGRAFÍA

<https://www.movetis.com/blog/las-7-mejores-apps-para-transportistas>

https://www.actiruta.com/Optimizador_de_rutas.aspx

<https://www.ontruck.com/>

<https://github.com/pepitz/petarOnTruck/tree/master>

<https://www.uber.com/es/es-es/>

<https://cabify.com/es>

https://oa.upm.es/67562/1/TFG_JOEL_VAL_GARCIA.pdf

11 ANEXOS

En caso de necesitar mostrar figuras (imágenes) grandes, planos, tablas muy extensas, encuestas, códigos...y cualquier otra información que exceda el texto normal del documento. Las tablas o figuras deben estar correctamente referenciadas e ir en su índice correspondiente, índice de tablas e índice de figuras, después del índice general.